

Kan Chiong Delivery Driver

You are given a city road network with time-dependent traffic obstructions.

Given:

- a start coordinate
- an end coordinate
- a departure time

Compute the **fastest route by travel time** and return:

1. total duration in seconds
2. arrival time
3. ordered list of traversed `edge_id` s

Please expose a `POST` endpoint on `/kan-cheong-delivery-driver`.

Batch Request Format

Each call to your endpoint sends **multiple test cases in one request**, as a JSON object mapping a caller-assigned case id to that case's input:

```
{
  "case_1": { "...": "one case's input, see Input Format below" },
  "case_2": { "...": "another case's input" },
  "...": "..."
}
```

Your endpoint must respond with a JSON object of the same shape - a map from the **same case ids** to that case's output (see Output Format below):

```
{
  "case_1": { "...": "your answer to case_1" },
  "case_2": { "...": "your answer to case_2" },
  "...": "..."
}
```

```
}
```

Cases are entirely independent of each other - solve and answer each one on its own. You do not need to answer cases in any particular order, but every case id present in the request must have a matching entry in your response.

Batch Example

Request:

```
{
  "case_1": {
    "start_coordinate": [0, 0],
    "end_coordinate": [1, 0],
    "start_time": "2026-06-10T08:30:00Z",
    "nodes": [[0, 0], [1, 0]],
    "edges": [
      { "edge_id": "edge_0", "node1": [0, 0], "node2": [1, 0], "base_durc
    ],
    "obstructions": []
  },
  "case_2": {
    "start_coordinate": [0, 0],
    "end_coordinate": [1, 0],
    "start_time": "2026-06-10T08:30:00Z",
    "nodes": [[0, 0], [1, 0]],
    "edges": [
      { "edge_id": "edge_0", "node1": [0, 0], "node2": [1, 0], "base_durc
    ],
    "obstructions": [
      {
        "edge_id": "edge_0",
        "edge": { "from": [0, 0], "to": [1, 0] },
        "start_time": "2026-06-10T08:00:00Z",
        "end_time": "2026-06-10T09:00:00Z",
        "speed_factor": 0.0
      }
    ]
  }
}
```

Response:

```
{
  "case_1": { "total_duration_sec": 60, "arrival_time": "2026-06-10T08:31
  "case_2": { "total_duration_sec": null, "arrival_time": null, "path": [
}
```

Timeout

You have **10 seconds** to respond to the entire batch request - regardless of how many cases it contains. This is a single hard cutoff on the whole request/response, not a per-case allowance, so a large batch requires an efficient solution, not just a correct one. If you do not respond within 10 seconds, the request times out and the **entire batch is scored as 0** - there is no partial credit for cases you'd already solved when the timeout hit.

Scoring

Each case in the batch is scored independently and correct answers add up across the batch - there's no partial credit within a single case, but solving more cases in the batch earns more points. Larger, more complex cases (more nodes/edges/obstructions) are worth more points than small ones, since solving them within the time limit requires an efficient solution, not just a correct one.

Input Format

The shape of a single case's value in the request map:

```
{
  "start_coordinate": [x, y],
  "end_coordinate": [x, y],
  "start_time": "ISO-8601",
  "nodes": [[x1, y1], [x2, y2], ...],
  "edges": [
    {
      "edge_id": "string",
```

```
    "node1": [x, y],
    "node2": [x, y],
    "base_duration_sec": 0
  },
  ],
  "obstructions": [
    {
      "edge_id": "string",
      "edge": {
        "from": [x, y],
        "to": [x, y]
      },
      "start_time": "ISO-8601",
      "end_time": "ISO-8601",
      "speed_factor": 0.0
    }
  ]
}
```

Notes

- Edges are **bidirectional** with the same base duration in both directions.
- Obstructions are **directional** and apply only when both match:
 - edge_id
 - edge.from -> edge.to

Output Format

The shape your answer for a single case must take (the value you put under that case's id in your response map):

```
{
  "total_duration_sec": 0,
  "arrival_time": "ISO-8601",
  "path": ["edge_id_1", "edge_id_2", "..."]
}
```

If destination is unreachable:

```
{
  "total_duration_sec": null,
  "arrival_time": null,
  "path": []
}
```

Constraints

- **No waiting** at nodes.
- Coordinates are integer pairs: `[x, y]` .
- `base_duration_sec` is an integer in `[0, 999]` .
- Cycles are allowed (a node may be revisited).
- If an obstruction becomes active during traversal, only the remaining untraveled portion is affected by the new `speed_factor` .
- `speed_factor = 0.0` means that directed traversal is blocked while active.

Examples

Each example below shows a single case's input/output - the shape of one value inside the batch request/response map described above, not a full batch request on its own.

Example 1

Input

```
{
  "start_coordinate": [0, 0],
  "end_coordinate": [3, 1],
  "start_time": "2026-06-10T08:30:00Z",
  "nodes": [[0, 0], [1, 0], [2, 0], [2, 1], [3, 1]],
  "edges": [
    { "edge_id": "edge_0", "node1": [0, 0], "node2": [1, 0], "base_durati
    { "edge_id": "edge_1", "node1": [1, 0], "node2": [2, 0], "base_durati
    { "edge_id": "edge_2", "node1": [2, 0], "node2": [2, 1], "base_durati
    { "edge_id": "edge_3", "node1": [2, 1], "node2": [3, 1], "base_durati
```

```

    { "edge_id": "edge_4", "node1": [1, 0], "node2": [2, 1], "base_durati
  ],
  "obstructions": [
    {
      "edge_id": "edge_1",
      "edge": { "from": [1, 0], "to": [2, 0] },
      "start_time": "2026-06-10T08:00:00Z",
      "end_time": "2026-06-10T09:00:00Z",
      "speed_factor": 0.5
    },
    {
      "edge_id": "edge_2",
      "edge": { "from": [2, 1], "to": [2, 0] },
      "start_time": "2026-06-10T08:15:00Z",
      "end_time": "2026-06-10T08:45:00Z",
      "speed_factor": 0.0
    }
  ]
}

```

Output

```

{
  "total_duration_sec": 230,
  "arrival_time": "2026-06-10T08:33:50Z",
  "path": ["edge_0", "edge_4", "edge_3"]
}

```

Explanation

edge_4 is preferred over edge_1 + edge_2 because of active obstruction impact.

Example 2

Input

```

{
  "start_coordinate": [0, 0],
  "end_coordinate": [3, 3],
  "start_time": "2026-06-10T08:30:00Z",
  "nodes": [[0, 0], [1, 0], [2, 0], [2, 1], [3, 1]],

```

```

"edges": [
  { "edge_id": "edge_0", "node1": [0, 0], "node2": [1, 0], "base_durati
  { "edge_id": "edge_1", "node1": [1, 0], "node2": [2, 0], "base_durati
  { "edge_id": "edge_2", "node1": [2, 0], "node2": [2, 1], "base_durati
  { "edge_id": "edge_3", "node1": [2, 1], "node2": [3, 1], "base_durati
  { "edge_id": "edge_4", "node1": [1, 0], "node2": [2, 1], "base_durati
],
"obstructions": [
  {
    "edge_id": "edge_1",
    "edge": { "from": [1, 0], "to": [2, 0] },
    "start_time": "2026-06-10T08:00:00Z",
    "end_time": "2026-06-10T09:00:00Z",
    "speed_factor": 0.5
  },
  {
    "edge_id": "edge_2",
    "edge": { "from": [2, 1], "to": [2, 0] },
    "start_time": "2026-06-10T08:15:00Z",
    "end_time": "2026-06-10T08:45:00Z",
    "speed_factor": 0.0
  }
]
}

```

Output

```

{
  "total_duration_sec": null,
  "arrival_time": null,
  "path": []
}

```

Explanation

end_coordinate is unreachable, so the expected result is the null-duration no-path response.

Example 3 (No Waiting + Cycling)

Input

```

{
  "start_coordinate": [0, 0],
  "end_coordinate": [2, 0],
  "start_time": "2026-06-10T08:30:00Z",
  "nodes": [[0, 0], [1, 0], [2, 0]],
  "edges": [
    { "edge_id": "edge_0", "node1": [0, 0], "node2": [1, 0], "base_durati
    { "edge_id": "edge_1", "node1": [1, 0], "node2": [2, 0], "base_durati
    { "edge_id": "edge_2", "node1": [0, 0], "node2": [2, 0], "base_durati
  ],
  "obstructions": [
    {
      "edge_id": "edge_1",
      "edge": { "from": [1, 0], "to": [2, 0] },
      "start_time": "2026-06-10T08:30:10Z",
      "end_time": "2026-06-10T08:30:20Z",
      "speed_factor": 0.0
    },
    {
      "edge_id": "edge_1",
      "edge": { "from": [1, 0], "to": [2, 0] },
      "start_time": "2026-06-10T08:30:30Z",
      "end_time": "2026-06-10T08:30:40Z",
      "speed_factor": 0.0
    },
    {
      "edge_id": "edge_2",
      "edge": { "from": [0, 0], "to": [2, 0] },
      "start_time": "2026-06-10T08:30:00Z",
      "end_time": "2026-06-10T08:32:00Z",
      "speed_factor": 0.2
    }
  ]
}

```

Output

```

{
  "total_duration_sec": 60,
  "arrival_time": "2026-06-10T08:31:00Z",
  "path": ["edge_0", "edge_0", "edge_0", "edge_0", "edge_0", "edge_1"]
}

```


Explanation

No waiting is allowed, so the route cycles on `edge_0` until `edge_1`'s blocking window clears.

Example 4 (No Waiting + Blocked at Start)

Input

```
{
  "start_coordinate": [0, 0],
  "end_coordinate": [1, 0],
  "start_time": "2026-06-10T08:30:00Z",
  "nodes": [[0, 0], [1, 0]],
  "edges": [
    { "edge_id": "edge_0", "node1": [0, 0], "node2": [1, 0], "base_durati
  ],
  "obstructions": [
    {
      "edge_id": "edge_0",
      "edge": { "from": [0, 0], "to": [1, 0] },
      "start_time": "2026-06-10T08:00:00Z",
      "end_time": "2026-06-10T09:00:00Z",
      "speed_factor": 0.0
    }
  ]
}
```

Output

```
{
  "total_duration_sec": null,
  "arrival_time": null,
  "path": []
}
```

Explanation

Waiting is not allowed, and the only outgoing move from `start_coordinate` is blocked (`speed_factor=0.0`) at departure time, so no valid route exists.

